

Recent Kolmogorov–Arnold Network (KAN) Research — Supplement

Model structure and core ideas, three papers (full text reviewed), math-free version

Prepared by: Mona Sharifi, PhD Candidate, Electrical Engineering, CUNY Graduate Center / CCNY

Date compiled: August 5, 2026

Purpose and Scope

This is a revised version of the earlier supplement covering the same three full-text papers, rewritten to strip out equations and formal notation and focus instead on model structure, architectural ideas, and what each paper actually changes about KANs conceptually. Quantitative results (accuracy numbers, speedups) are kept where they help convey how significant a result is, but described in plain language rather than as formulas.

Papers covered:

- Faroughi, Mostajeran, Mashhadzadeh & Faroughi — "Kolmogorov-Arnold networks for data-driven, physics-informed, and deep-operator learning: a review, synthesis, and new analysis," *Neural Networks* 200 (2026) 108791.
- Zhang, Fan & Lu — "RSS-KAN: Replacing B-splines with ReLU, Sigmoid, and Sine for accurate and efficient Kolmogorov–Arnold Networks," *Neurocomputing* 672 (2026) 132796.
- Wolff, Alesiani, Duhme & Jiang — "Clifford Kolmogorov-Arnold Networks," arXiv:2602.05977v2 (revised June 23, 2026).

1. Faroughi et al. — KAN Review, Synthesis, and New Analysis

Neural Networks 200 (2026) 108791 — Full text reviewed

DOI: <https://doi.org/10.1016/j.neunet.2026.108791>

What the paper is

A large review article that organizes essentially all recent KAN research into three families: data-driven KANs (trained purely on input-output data, like a regular regression network), physics-informed KANs or PIKANs (trained to also satisfy a known differential equation, the KAN version of a PINN), and deep-operator KANs or DeepOKANs (trained to learn an entire family of solutions at once, the KAN version of DeepONet). Alongside summarizing the literature, the authors also ran their own new comparison experiments pitting KAN-based models against standard MLP-based models on the same tasks.

The core idea of a KAN, as explained in the paper

A standard neural network (MLP) puts the learnable part on the connections between neurons as fixed weights, and puts a fixed, non-learnable nonlinearity (like ReLU) at each neuron. A KAN flips this: the connections carry learnable curved functions instead of fixed weights, and the neurons just add things up. In other words, instead of learning how much to weight each input, a KAN learns the actual shape of the function applied to each input before those results are summed. The paper frames this as more mathematically grounded, since it directly mirrors a classical theorem (the Kolmogorov-Arnold theorem) about representing any multi-variable function as a sum of simpler single-variable functions. Practically, this shape-based structure is also why KANs are described as more interpretable than MLPs: you can look at each edge's learned curve and see what it's doing, rather than reading an opaque weight matrix.

The original KAN implementation represents each edge's learned curve using B-splines (smooth, piecewise polynomial curves, the same kind of curve used in CAD software or font design). Later variants, discussed throughout the review, swap this B-spline curve for other curve types — Chebyshev polynomials, wavelets, radial basis functions (bell-shaped bumps) — each with different trade-offs in smoothness, parameter count, and how well they capture sharp or oscillatory behavior.

A KAN can also be stacked in layers just like an MLP, and the paper notes that shallow (2-layer) KANs, while mathematically the "pure" form of the underlying theorem, are often not expressive enough in practice, so most modern KANs use multiple layers of these learned-curve connections.

Data-driven KANs — how they're trained and what the comparisons showed

In the purely data-driven setting, a KAN is just trained like any regression model: feed it inputs, compare its output to the true output, and adjust the learned curves to reduce the error. The paper's own new experiments used a KAN with Chebyshev-polynomial edges (called cKAN) to predict how a granular material (like sand) deforms under stress, comparing it against a matched MLP.

Reported takeaways from that comparison:

- The KAN version was noticeably more accurate at predicting stress under a loading path it had never seen during training, with roughly 3 to 6 times lower error than the MLP depending on the exact setup.
- During training, the KAN's error dropped steadily and kept improving, while the MLP's error dropped fast at first and then plateaued early — described as the KAN continuing to learn where the MLP stalls.
- The KAN version took somewhat longer per training step than the MLP (roughly 50-60% more time per epoch), because each learned curve is more expensive to evaluate than a simple weighted sum. So the KAN's edge in this experiment was pure accuracy and continued learning, not raw speed.
- The authors also looked at a technical training-dynamics diagnostic (essentially, how "well-behaved" the optimization landscape is during training) and found the KAN's version was more stable and consistent than the MLP's across ten different random starting points.

Physics-informed KANs (PIKANs) — the idea

A PIKAN is a KAN that's trained not just to match data, but to also satisfy a known physics equation (a PDE) as an extra training objective — the same idea as a Physics-Informed Neural Network (PINN), just with a KAN instead of an MLP doing the learning. The network is fed spatial and time coordinates and asked to output a physical quantity like pressure, displacement, or temperature; during training, the model's own output is checked against the governing equation, and any violation of that equation is penalized just like a normal prediction error would be.

A key structural choice explored is how strictly to enforce boundary and initial conditions. In a "soft" version, violating the boundary condition is just penalized as part of the loss, so the network is only encouraged, not guaranteed, to get it right. In a "hard" version, the network's output is mathematically wrapped so it automatically satisfies the boundary condition no matter what the network predicts internally — a structural guarantee rather than a training incentive.

The paper's central practical finding here is about basis-function choice: switching a PIKAN's edge curves from B-splines to Chebyshev polynomials, with no other change, cut the error by roughly 90% on a benchmark PDE (the Allen-Cahn equation). Pairing that Chebyshev-based PIKAN with a more advanced optimizer (a quasi-Newton method called SSBroyden, instead of the usual Adam) pushed the error down by another 99% on top of that. Across the many comparisons collected in the review, PIKANs are generally reported to beat equivalent PINNs by 85-99% in error, and usually with fewer trainable parameters, not more.

The paper also describes a scaling trick (rescaling the input domain and the physics-residual terms to fit the natural range of the Chebyshev curves) that made a big difference on a harder, wider version of the Allen-Cahn problem — an 88-90% error reduction just from that rescaling step, with no architecture change.

Deep-operator KANs (DeepOKANs) — the idea

A normal PIKAN learns one specific solution for one specific set of boundary/initial conditions. A DeepOKAN instead learns an entire family of solutions at once — given a new initial condition or boundary condition it hasn't seen, it can produce the right solution without being retrained. Structurally, this is done with two sub-networks: a "branch" network that encodes the specific input condition (e.g., a crack shape, or an initial temperature profile), and a "trunk" network that encodes the query point (where in space/time you want the answer). Their outputs are combined to produce the final prediction. DeepOKAN is simply this same two-network structure, but built from KAN layers instead of MLP layers in both branches.

Reported comparisons against the MLP-based version of this idea (DeepONet) showed the KAN-based version was substantially more accurate in shallow (simple) architectures — over an 85% error reduction on a flow-through-porous-media benchmark — though the gap narrowed once both models were made deeper. The KAN version also held up noticeably better when noise was added to the input data, whereas the MLP version's accuracy degraded faster as noise increased.

Stated limitations of KANs, as discussed in the review

- KANs are reported to struggle, relative to MLPs, on functions with sharp jumps or discontinuities, and to be more sensitive to noisy data.
- Training a KAN can take substantially longer than training an MLP of similar size — the paper cites cases where it took up to two orders of magnitude (100x) longer, particularly for the original B-spline version.
- KANs perform worse than MLPs when there isn't much training data available.
- A structural/engineering complaint: KANs aren't natively supported by mainstream deep learning libraries the way MLPs are, and the original B-spline curve evaluation involves a step-by-step (recursive) calculation that doesn't parallelize well on GPUs. This is exactly the practical problem that the RSS-KAN and Clifford-KAN papers below are each trying to solve in different ways, though this review doesn't cite either of them directly.

Relevance to KAN-SMLM work

Not stated in the paper — this is the report author's own read. The recurring theme that swapping B-splines for a different curve type (Chebyshev, in this paper's own experiments) can meaningfully improve both accuracy and training stability, without changing the overall network architecture, is the most transferable idea here: it suggests the choice of edge-curve type is a real design lever worth considering for the KAN-SMLM localizer, separate from anything about depth or width.

2. Zhang, Fan & Lu — RSS-KAN

Neurocomputing 672 (2026) 132796 — Full text reviewed

DOI: <https://doi.org/10.1016/j.neucom.2026.132796>

The core idea

This paper's starting complaint is specifically about the B-spline curve used in the original KAN: B-splines are computed through a step-by-step recursive process that doesn't run efficiently on GPUs, and their shape is somewhat rigid (each one is just a shifted, identically-shaped bump). Their proposed fix, RSS-KAN, replaces the B-spline on every edge with a custom-built curve that blends three familiar, simple functions: a ReLU (which acts like a window, turning the curve on and off over a specific local range), a Sigmoid (a smooth S-shaped curve that controls how the curve rises and levels off), and a Sine wave (which lets the curve wiggle periodically). Each edge learns its own mixing weights for these three ingredients, so different edges can end up looking mostly ReLU-like, mostly smooth, mostly wavy, or some blend, depending on what the training data needs.

The motivation for choosing exactly these three: the ReLU part gives "local support" — the curve only does something over a bounded window, similar to how a B-spline works, which keeps changes in one part of the input space from messing with other parts. The Sigmoid part gives smooth shape control, letting the curve's peak height

and steepness adapt. The Sine part lets the curve capture high-frequency, oscillating, or periodic patterns that a purely smooth curve (or a purely ReLU-based curve) would struggle to fit tightly.

Crucially, all three of these ingredient functions can be computed using ordinary matrix operations — no recursion, no step-by-step evaluation like B-splines require. That's the paper's central engineering claim: this hybrid curve keeps the flexibility advantage of a rich, multi-part basis function while staying fully GPU-friendly.

The paper also proves, in a fairly plain-language sense, that this hybrid curve is strictly more expressive than using any one of its three ingredients alone — a network built only from ReLU, only from Sigmoid, or only from Sine can each be reproduced as a special case of RSS-KAN by just turning the other ingredients' weights to zero. So nothing is lost by using the combined version; it can only do at least as much as any single-ingredient version.

How well it worked, and against what

The authors tested RSS-KAN against five other improved KAN variants (each of which had already tried some other fix to the B-spline problem): BSRBF-KAN (blends B-splines with bell-shaped RBF curves), Efficient-KAN, Fast-KAN (swaps B-splines for a simpler bell-curve family), ReLU-KAN (the direct predecessor, using ReLU alone), and SineKAN (using Sine alone). The comparison used eight test functions of varying character — smooth, oscillating, high-dimensional, and combinations of these.

Reported accuracy results: RSS-KAN had the lowest error on six of the eight test functions, and specifically won by a wide margin on the functions with strong oscillation or asymmetry — exactly the cases its Sine ingredient is meant to help with. On two of the smoothest test functions, a competing model (Efficient-KAN, which uses smooth bell-shaped curves well-suited to gentle, structured functions) edged it out slightly. Averaged across the whole test set, the improvements over the other models ranged from roughly one order of magnitude (10x) up to three to five orders of magnitude (thousands to hundreds of thousands of times lower error), depending on which competitor and which test function.

Reported speed results: the original recursive B-spline KAN was by far the slowest model tested, as expected. RSS-KAN was not the fastest model overall — it was roughly twice as slow as the fastest competitors (Fast-KAN and ReLU-KAN) per training step, because blending three ingredients costs more than using just one. But it was in the same speed range as BSRBF-KAN and Efficient-KAN, and still 5 to 15 times faster overall than the original B-spline KAN. The paper argues that because RSS-KAN needs fewer training steps to reach a given accuracy, its total training time ends up competitive with the fastest models even though each individual step costs a bit more.

What the follow-up experiments showed about why it works

A component-removal (ablation) study tested every combination of the three ingredients alone and in pairs, to check whether the full three-way blend is actually necessary or if it's just extra complexity. Using only ReLU worked fine on functions with sharp corners but did poorly on periodic functions. Using only Sigmoid worked for smooth, gently-curving functions but struggled with local oscillation. Using only Sine was excellent for periodic functions but converged slowly on non-periodic ones. Every two-ingredient pairing did better than either ingredient alone, and the full three-ingredient version did best or nearly best across the board — supporting the idea that the three pieces genuinely complement rather than duplicate each other. The authors also tried adding a fourth ingredient (Tanh) on top of all three, which did not help and sometimes hurt performance slightly, suggesting three well-chosen ingredients is close to the sweet spot rather than "more is always better."

The paper also tracked how the mixing weights for the three ingredients evolve during training, and found the network tends to quickly figure out which ingredient should dominate for a given task (e.g., leaning hard into Sigmoid for a smooth transition, or into Sine for a periodic pattern) in the early stages of training, then fine-tunes the balance later as the model refines its fit. Deliberately removing or perturbing one ingredient's contribution after training showed that the "dominant" ingredient for a given task mattered the most, but the other two still played a real, measurable supporting role rather than being dead weight.

Real-world test

Beyond the synthetic test functions, the authors tried RSS-KAN on a genuine weather dataset, predicting air temperature from other measured weather variables over a 24-hour sliding window. RSS-KAN produced lower

training and validation error than every comparison model, including a standard MLP, and its predicted temperature curve tracked the true curve more closely, especially around the daily peaks and troughs where competing models tended to lag behind or overshoot.

Relevance to KAN-SMLM work

Not discussed in the paper — this is the report author's own read. The most directly transferable point is the engineering one: since your existing pipeline uses B-spline KAN layers, and this paper's whole argument is that B-splines specifically are the source of the GPU-inefficiency problem, a ReLU-Sigmoid-Sine hybrid curve is presented as a straightforward drop-in replacement if training speed on the localizer ever becomes a bottleneck. The finding that a curve blending local support, smoothness, and periodicity outperforms any single one of those properties alone might also be worth keeping in mind if PSF sidelobe structure (which is itself somewhat oscillatory) becomes something the localizer needs to fit tightly — though the paper itself never touches on imaging or microscopy.

3. Wolff, Alesiani, Duhme & Jiang — Clifford Kolmogorov-Arnold Networks

arXiv:2602.05977v2 — Full text reviewed (revised June 23, 2026)

Link: <https://arxiv.org/abs/2602.05977>

The core idea

This paper generalizes KANs to work with a broader kind of number than the usual real or complex numbers. A prior paper by some of the same authors had already extended KANs to complex numbers (CVKAN), useful for problems like signal processing where quantities naturally have both a magnitude and a phase. This new paper goes a step further, extending KANs to Clifford algebras — a mathematical framework that generalizes complex numbers and quaternions into a much richer family of "multi-part" numbers capable of representing not just points, but also directions, rotations, areas, and volumes, all within one unified number system. The motivating examples given are things like electromagnetic fields, 3D geometry, and robot arm movement, where a single physical quantity naturally has several intertwined components that a plain real or complex number can't capture cleanly.

Structurally, the idea is straightforward given the CVKAN precedent: instead of each KAN edge learning a curve that maps a real number to a real number (or a complex number to a complex number), each edge in a Clifford-KAN learns a curve that maps one of these richer multi-part numbers to another multi-part number, letting the network learn transformations that respect the internal structure of the input (for example, correctly handling how a rotation and a direction interact, rather than treating each numeric component independently).

Each edge's curve is still built the same conceptual way as prior KAN variants: a set of bump-shaped basis functions (RBFs, the same bell-curve idea as FastKAN) placed on a grid, each with a learnable weight, and the curve's output is the weighted sum of these bumps. The novelty is that both the grid points and the weights now live in this richer Clifford-number space rather than being plain real numbers.

The main problem this paper had to solve: too many grid points

The natural way to build this grid — spacing points evenly across every dimension of the Clifford-number space — runs into a serious scaling problem. As you move to richer number systems (e.g., quaternions, which have four components instead of two like complex numbers), the number of grid points needed to cover the space evenly grows exponentially. The paper gives a concrete example: with a modest 8 points per dimension, a quaternion-based network needs over 4,000 grid points, each with its own learnable weight, just for a single edge. This is described as the central obstacle to making higher-dimensional Clifford-KANs practical.

Their proposed fix is to abandon the evenly-spaced grid entirely and instead scatter the grid points using a particular kind of structured randomness (a "Sobol sequence," a well-known technique from numerical simulation for spreading sample points evenly through a space without needing exponentially many of them). The paper shows, with a supporting mathematical argument, that this Sobol-based grid gives an unbiased approximation of the same

target that the full evenly-spaced grid would give, and that its accuracy improves in a predictable, well-behaved way as more points are added — without ever needing the full exponential grid size.

Normalization: keeping the network well-behaved during training

Because the network's grid only covers a fixed range, the paper also had to make sure values flowing between layers don't drift outside that range and end up far from every grid point (which would make the learned curve useless for those inputs). Three different ways of normalizing the data between layers were tried: normalizing each individual component of the multi-part number across the whole layer, normalizing each full multi-part number as a unit, or normalizing each component of each individual node separately. A fourth option — doing no normalization at all — was also tested as a baseline.

How well it worked

On the same simple test functions used to validate the earlier complex-number version (CVKAN), the new Clifford-KAN matched CVKAN's performance when using the full, evenly-spaced grid, confirming the extension didn't lose anything. The Sobol (randomly-scattered) grid's accuracy improved steadily as more points were added, and by around 8 points per dimension it essentially caught up to the full grid — while needing dramatically fewer parameters to get there in higher-dimensional settings.

On a larger, more realistic dataset (100,000 samples related to holography), the full-grid version of Clifford-KAN performed close to CVKAN, and a Sobol grid using only about three-quarters of the full grid's points achieved nearly identical accuracy — evidence that the random-grid trick doesn't cost much accuracy even on a real dataset, not just toy problems. Grids that were too sparse (very few points per dimension) failed to learn the function well, so there's a floor below which the savings stop being worth it.

When they tested genuinely higher-dimensional Clifford number systems (including a quaternion-like system and a couple of others used in computer vision and robotics), the Sobol grid generally matched the full grid's accuracy while using a small fraction of the parameters — in one specific case, matching or even beating the full grid's accuracy while using only about 2% of its parameter count. This is presented as the paper's strongest result: the parameter-saving trick doesn't just avoid hurting accuracy, it can sometimes help, though this benefit was inconsistent across every number system and dataset tested.

On the normalization comparison, doing no normalization at all was clearly the worst option by a wide margin, while the three actual normalization methods all performed similarly to each other — the paper's conclusion is that some form of normalization is essential, but which specific type you pick doesn't matter very much.

Stated limitations and open issues

- Both CVKAN and the new Clifford-KAN were reported to be quite sensitive to how the network's weights are initialized before training; the authors had to increase the learning rate substantially compared to the earlier CVKAN paper to get more consistent, competitive results, and note that some instability across different training runs still remains even with this fix.
- One of the four test functions (a function with especially large swings in its output values) was reported to be far harder to fit well than the others, for both models.
- The random Sobol grid didn't always win — for some specific combinations of number system, dataset, and grid density, it performed worse than the full grid, with notably high run-to-run variability in those cases. The authors describe this as an open, not fully resolved, practical inconsistency rather than a clean win in every setting.

Relevance to KAN-SMLM work

Not discussed in the paper — this is the report author's own read. The most plausible point of contact is conceptual rather than direct: the paper's finding that having some form of normalization matters far more than which specific type you choose echoes the same territory as your own LayerNorm fix for mode collapse in V10, fixed, and might be worth keeping in mind if further normalization tuning comes up. The core idea of mapping KAN edges to richer, multi-part number outputs isn't obviously something to bring into 2D/3D coordinate regression for SMLM unless

DH-PSF orientation or phase information were deliberately reframed as one of these multi-part numbers — which the paper never suggests, and which would be a genuinely new idea on your part rather than something drawn from the text.

Summary Table

Paper	Venue / Date	Core idea	Confidence
Faroughi et al. — KAN review & synthesis	Neural Networks, Mar 2026	Surveys data-driven/physics-informed/operator-learning KANs; new experiments show swapping B-splines for Chebyshev curves boosts accuracy and stability	
Zhang, Fan & Lu — RSS-KAN	Neurocomputing, Jan 2026	Replaces B-spline edge curves with a learnable ReLU+Sigmoid+Sine blend; fully matrix-computable, much faster and more accurate	
Wolff et al. — Clifford KAN	arXiv, rev. Jun 2026	Extends KAN edges to output richer multi-part (Clifford-algebra) numbers; uses randomly-scattered grid points to avoid exponential parameter growth	

All three entries above are full-text reviewed. This version replaces the earlier math-heavy supplement; use whichever version fits the audience.

Notes on This Report

- This version deliberately omits equations and formal notation in favor of plain-language description of model structure, architectural choices, and reported results.
- Numerical results (accuracy percentages, speedups) are kept because they convey the scale of each finding, but are described narratively rather than in table-of-formulas form where possible.
- The original PDFs remain the authoritative source if precise mathematical notation is needed for citation in a qualifying-exam or publication context.